

Lab 9 — Orchestrated Batch Scoring

Track B (BMD-45 vehicle detection) · Week 9 · DS5619 Machine Learning Systems Operations

Setup

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python generate_for_student.py --student-id <your roll number or institute email>
```

Record your `--student-id` value in `NOTES.md`.

Files

- `src/mini_orchestrator.py` — implement the three `# TODO` functions.
- `src/mock_detector.py` — dependency-light stand-in detector (provided, don't edit).
- `src/run_pipeline.py` — complete driver script, defines and runs the actual DAG. Don't edit.
- `data/fixtures/` — your personalized sample images to batch-score, generated above (don't hand-edit).

Background

The batch-scoring DAG has this shape (this is the "anatomy of a DAG" from the lecture, applied):

```
extract --> score --> load
                \-> notify
```

- `extract`: list every image under `data/fixtures/`.
- `score`: run the detector over each image, accumulate detection counts.
- `load`: write `batch_report.json` — depends only on `score`.
- `notify`: a deliberately FLAKY task (fails twice, then succeeds) standing in for something like a webhook call — also depends only on `score`. It exists so your retry logic actually gets exercised by a real DAG run, not just by unit tests.

Your task

`src/mini_orchestrator.py` (three functions marked `# TODO`, each has a full docstring spec, ~45 min)

- `topological_order(tasks)` — order tasks so every task runs after its dependencies (Kahn's algorithm); detect cycles and missing dependencies.
- `run_task_with_retry(task, context)` — retry a single task up to `max_retries` additional times before giving up.

- `run_dag(tasks, context)` — run every task in dependency order, sharing one `context` dict between them, stopping if a task ultimately fails.

```
python src/run_pipeline.py
```

Defines the actual 4-task DAG above and runs it, writing `batch_report.json` and `dag_run_summary.json` (`src/run_pipeline.py` is complete, don't edit it).

Self-check

```
pytest tests/ -q
```

This is a self-check, not the grader.

Deliverables (what you commit)

- `src/mini_orchestrator.py`, completed.
- `batch_report.json` and `dag_run_summary.json` (from running the pipeline).
- A short `NOTES.md`: the `--student-id` value you used (required — see above), how many attempts did `notify` take in your run (check `dag_run_summary.json`), why does `load` still succeed even though `notify` — its "sibling" in the DAG — failed on its first two attempts? What does that tell you about how failures in one branch of a DAG should (or shouldn't) affect an unrelated branch? And — tying back to this week's FinOps content — if `notify` were a metered API call you paid for per attempt, what would you change about the retry strategy, and why is "retry every failed task the same way" a risky default once cost enters the picture?

Grading checklist

- ☐ `data/fixtures/` matches what `generate_for_student.py --student-id <NOTES.md value>` actually produces.
- ☐ `topological_order` correctly handles a branching DAG (not just a linear chain) and raises `ValueError` on both a cycle and a missing dependency.
- ☐ `run_task_with_retry` retries the correct number of times and re-raises the last exception if a task never succeeds.
- ☐ `run_dag` shares context correctly between tasks and stops running downstream tasks after an unrecoverable failure (checked with a held-out DAG, not just the provided one).
- ☐ The real pipeline run shows `notify` succeeding after retries in `dag_run_summary.json`.
- ☐ `NOTES.md` shows real reasoning about branch independence and the retry-cost question, not just a restatement of the DAG shape.
- ☐ Meaningful commit history and a working README.

Submission

```
git add -A  
git commit -m "Week 9: orchestrated batch scoring"  
git tag week09-submit  
git push origin main --tags
```